

MessageFoundry — A Mental Model of the Project

Early Access · MessageFoundry v0.3.2 · September 2026

Open-source healthcare interface engine · Python · v0.3.2 · prepared 2026-06-18, revised 2026-07-30

This guide explains MessageFoundry's building blocks, message flow, required invariants, and repository layout. Use it to understand the codebase, then follow the linked references for operating details.

1. What it is, in one breath

MessageFoundry is an **open-source Python interface engine for healthcare**. It receives, routes, transforms, and validates clinical and business messages between systems. It handles **HL7 v2.x by default** and accepts other payloads, including JSON, XML/SOAP, X12 EDI, and database records.

Use guided wizards to create connections and routes, validate configuration on save, and test sample messages. Write custom routing and handling in Python. Both approaches produce version-controlled Python. Connection settings can also live in a TOML file, edited by hand or through the VS Code interface.

The engine includes a durable queue without a separate broker, using SQLite by default or PostgreSQL/SQL Server. Authentication, role-based access, audit records, and encryption at rest support its handling of healthcare data.

Stack: python-hl7 provides tolerant parsing, and hl7apy provides strict validation. FastAPI/uvicorn serves the engine API. SQLite/aiosqlite, PostgreSQL, and SQL Server provide store options. The web console uses `/ui` and `messagefoundry_webconsole`. PySide6 supports the standalone test harness. The engine uses Python 3.14+ and asyncio.

2. The core model: a graph of four building blocks

Configuration is a **graph wired by name**. Inbound Connections name Routers, Routers name Handlers, and Handlers send to outbound Connections. There is no enclosing "channel" object. "Channel" and "route" can describe a connected path in prose, but neither names a built configuration element.

Connection	An endpoint that receives (inbound) or sends (outbound) messages — MLLP, file, TCP, HTTP/REST, SOAP, database, SFTP/FTP. Every message in or out is counted and logged.	inbound() / outbound() factory, or connections.toml	transports/
Router	A pure Python function bound to ONE inbound. Sees every received message. Returns the name(s) of Handler(s) to forward to. May filter (return none).	@router function	config modules
Handler	A pure Python function taking a message from a Router: filter → transform → return Send(s) to one or more outbound Connections.	@handler function returning Send	config modules
Message store	Durable persistence + the staged queue for received/processed/errored messages. SQLite (WAL) by default. Postgres & SQL Server for production.	(<i>infrastructure</i>)	store/

The wiring, drawn:

```

inbound Connection —names→ Router —names→ Handler —Send→ outbound Connection
(MLLP :2600)           (@router)           (@handler)           (MLLP host:port)

```

every hop is by NAME; the whole config is a flat, by-name graph – not a nested 'channel' that owns its pieces.

Why “wired by name,” and why it is a graph

“Wired by name” means the links between pieces are just **strings, resolved when the config loads**:

- An inbound names its Router (router="acme_adt_router").
- A Router returns its Handlers' names (["acme_adt_handler"]).
- A Handler returns Send("OB_ACME_ADT", msg) naming an outbound.

The loader resolves names into a Registry that the engine runs. Connections, Routers, and Handlers form a graph with named links. One Router can select many Handlers, and many Handlers can send to one outbound.

The named graph supports reuse, testing, and configuration tools:

- **Reuse instead of duplication.** Define a destination, a shared transform, or a router once and reference it from anywhere. A bundled “channel” would force you to copy a shared transform or destination into every channel that needs it (the exact pain ADR 0007 calls out). A name graph just adds an edge.
- **Each node has a testable contract.** A Router or Handler receives a message and returns names or Sends. Test each function independently. Teams can develop separate functions at the same time under the modularity standard (\$10).

- **Names let code and data mix.** The loader turns Python and connections.toml definitions into identical Registry entries through the same factories. This translation is called desugaring. A wizard can add or reconnect a Connection by name without changing Router or Handler code.
- **The graph maps straight onto the runtime.** Each node becomes its own supervised asyncio worker, and the edges are the staged-queue handoffs (ingress → routed → outbound, \$5). A slow or failing node does not block its siblings. You restart or scale one node, not a monolith.
- **The loader checks the graph.** It rejects unknown Routers, missing Handlers, duplicate names, and duplicate ports at load or check time. Tools can draw the graph from those names (docs/architecture-diagram.md).
- **Rewire by changing names.** Add a destination with another Send, or define an outbound and link to it. Each change remains a version-controlled diff.

Mental model: picture the config as a wiring diagram, not a stack of self-contained channels. The boxes (Connections / Routers / Handlers) are each defined once. The arrows are names. To understand a feed, follow the names. To change it, change an arrow.

The simplest real route (from samples/config)

Receive ACME ADT over MLLP and forward it to an environment-specific downstream. This is a complete, runnable MessageFoundry config module:

```
from messagefoundry import MLLP, Send, env, handler, inbound, outbound, router

inbound("IB_ACME_ADT", MLLP(port=2600), router="acme_adt_router")
outbound("OB_ACME_ADT", MLLP(host=env("acme_adt_host"), port=env("acme_adt_port", cast=int)))

@router("acme_adt_router")
def route(msg):
    return ["acme_adt_handler"]      # decide where it goes

@handler("acme_adt_handler")
def handle(msg):
    # filter / transform here
    return Send("OB_ACME_ADT", msg)  # deliver to the outbound
```

The env() function resolves the downstream peer from environments/<env>.toml at load time. The **same module runs unchanged in dev, staging, and prod**. Names use [TYPE]_[PARTNER]_[MESSAGE], such as IB_ACME_ADT for an inbound and OB_ACME_ADT for an outbound.

3. The architecture: engine-as-library + web-console-over-API

The engine and its clients communicate through one API:

- **Engine** — a headless asyncio service (FastAPI/uvicorn). It owns the store and supervises one runner per inbound connection. **No GUI imports** — it is testable headless and runnable as a Windows service.

- **Web console** — the operator UI, a browser SPA the engine serves same-origin at `/ui` (`messagefoundry_webconsole`, mounted in-process — ADR 0065). It talks to the engine ONLY over the localhost HTTP/WebSocket API, never importing the engine or touching the DB directly. It is the **sole operator console** — the former PySide6 desktop console was retired (BACKLOG #103). PySide6 now backs only the standalone test harness.

Dependency direction (one-way — never violate): pipeline / transports / parsing / store / config never import api. The API depends on the engine. The clients (web console, harness) depend on the API. One carve-out: parsing/ is a pure HL7 library a client may import for client-side rendering (e.g. the harness's Parse Tree view). Importing any other engine package from a client is forbidden.

Why it matters: the deployment split (in-process / local daemon / remote host) becomes a **config choice, not an architectural fork**. The same API path serves all three. The database holds runtime state and messages only — **never configuration**.

What this split buys you

The headless engine and API contract support these uses:

- **The API supports different clients.** Clients include the web console, VS Code extension, command-line tool, test harness, scripts, and monitoring systems. Each uses the HTTP/WebSocket API in `api/app.py`. A new interface can use that contract without changes to the engine.
- **The same engine supports different deployments.** A Python application can import it, or a Windows service can run it headlessly. The browser console connects to a local daemon's `/ui`. Remote TLS access uses the same API. These deployment choices do not require separate engine editions.
- **The engine needs no display.** Engine packages do not import the GUI. The engine can run in CI, a container, or an unattended service. Message flow does not depend on an open window.
- **The API enforces access controls.** Authentication, role-based access, audit, and TLS apply at the API. The console authenticates like other clients, and PHI access records identify the acting user. Clients must not import engine packages or access the database directly.
- **Engine and console releases are independent.** Teams can build, test, and release each component against the API contract (§10). Each component keeps its own internal model. The engine mounts the web console from a separately versioned wheel.
- **Clients attach and detach freely.** Reload the web console without touching the engine (and vice versa). Run several observers against one engine. Receive live push updates over the WebSocket stats feed (WS `/ws/stats`) — all without interrupting message flow.

The engine exposes a published service contract. Every user interface, including the operator console, is a client of that service. The API provides access controls and supports embedded, local, and remote deployments.

Configuration lives in files, not the database

Configuration and runtime data have separate homes:

Where it lives	An org-owned, version-controlled config repo (the --config dir), plus a per-instance messagefoundry.toml and MEFOR_* env vars for operational settings.	The store — SQLite, PostgreSQL, or SQL Server.
What it holds	The message graph and operational settings define connections, routing, and transforms. Files contain Connections, Routers, Handlers, connections.toml, code sets, and environments/\<env>.toml.	Mutable runtime state <i>only</i> : received/processed messages, the queue’s stage rows, audit, and delivery bookkeeping.

The database never holds configuration. The config repo never holds PHI or secrets — secrets come from MEFOR_* environment variables, injected per instance.

This split separates change review from stored message data:

- **Git records configuration changes.** Each diff has an author and history. Teams can review or reverse changes and compare environments. This is harder when configuration is an opaque database object.
- **The database does not supply executable logic.** Routers and Handlers are Python code. Database write access must not permit changes to that code. Configuration reaches production through pull-request review, `messagefoundry check`, and a signed, pinned wheel (ADR 0017). Store encryption, access controls, and audit protect message data separately.
- **State and logic have independent blast radius and restore paths.** Restore or corrupt the database and you have not touched routing. Promote new config and you have not touched a single stored message. Disaster recovery is two clean, independent operations: redeploy config from git, restore data from a DB backup.
- **One source of truth, identical across environments.** The same modules run unchanged everywhere. Only environments/\<env>.toml differs (dev vs prod peers and hosts). Git is the source of truth — no “someone changed it in the prod GUI, and now staging ≠ prod” drift.

The configuration repository is the source for deployed logic. Database backups preserve runtime state and messages. Review and version configuration independently from database changes.

4. The tools: what is in the box

The toolkit includes the engine and separate clients connected through its local API (§3). Use the table to find each tool and its command:

Engine service	messagefoundry serve <i>(as a Windows service via NSSM)</i>	The headless runtime: owns the store, runs the Connection/Router/Handler graph through the staged queue, and exposes the localhost HTTP/WebSocket API. Everything else talks to this.
Command-line tool	messagefoundry \<cmd>	Commands include serve, init (create a config repo), and validate / graph / dryrun / check (the commit/CI gate). The connection command edits connections.toml. The generate command creates synthetic HL7. Other commands manage keys and audit records. The introspection commands touch no network — git hooks and the VS Code extension shell them.
Admin & monitoring web console	Open the engine's /ui . It is on by default . Disable it with <code>[security].serve_web_console = false</code> . It uses the <code>messagefoundry-webconsole</code> wheel.	The browser console shows connections, messages, dispositions, delivery and audit history, and an HL7 parse tree. It also supports dead-letter replay and user, session, and MFA management. This API client never accesses the database directly. The former PySide6 desktop console was retired (BACKLOG #103).
VS Code configuration extension	the ide/ extension (open in VS Code, press F5)	Includes a New Route Wizard, validation on save, and a Test Bench for .hl7 files with before/after comparisons. Stage → Promote deploys to a running engine. The @messagefoundry chat participant provides HL7-aware AI assistance. Shells the CLI's introspection commands.
Test harness	python -m harness <i>(standalone PySide6)</i>	Tests a running engine with synthetic, PHI-free traffic. Send, Receive, File, Compose, and Monitor tabs inject ACK faults, malformed messages, and delivery failures. Headless CI scenarios check dispositions. A separate asyncio load-testing engine runs configurable warmup, ramp, and soak profiles, then produces an SLO report.
Tee relay	python -m tee <i>(standalone, no engine imports)</i>	Receives traffic before the legacy engine and shadow MessageFoundry instance. It acknowledges receipt and forwards identical bytes to both engines for output comparison. To roll back, stop the relay. <i>Test/synthetic data only — not PHI-hardened.</i>

The throughline: the engine is the only long-running service and the only thing that touches the store. The web console (in a browser), the extension, and the harness drive or observe it over the one API. The tee relay sits in front of it. That is the “one contract, many clients” split from §3, made concrete.

5. How a message flows: the staged pipeline

The store uses a **generic staged queue** with a stage discriminator. **SQLite in WAL mode** is the default and requires no separate installation. PostgreSQL and Microsoft SQL Server provide server-database options. Each received message passes through three persisted stages with separate async workers (ADR 0001):

ingress	The raw message, committed before the ACK.	listener (<i>decode/parse/validate, sync</i>)	router worker (<i>1 per inbound</i>)
routed	One row per Handler the Router selected — carries the raw, awaiting transform.	router worker runs <code>route_only</code>	transform worker (<i>1 per inbound</i>)
outbound	One row per destination, ready to deliver.	transform worker runs <code>transform_one</code>	delivery worker (<i>1 per outbound</i>)

Separate routing and transformation workers prevent a slow transform from stopping routing. A slow Router or transform does not stop intake. Outbound workers drain independently, so a slow destination does not stop its siblings.

The reliability invariant (do not break)

At-least-once, broker-free: The transactional staged queue (SQLite in WAL mode, or PostgreSQL / SQL Server) gives at-least-once delivery, retries, replay, and dead-lettering WITHOUT a separate message broker. The inbound is ACKed only after the raw message is durably committed to the ingress stage (ACK-on-receipt). Every stage handoff (ingress→routed, routed→outbound) is a single committed transaction: claim → produce next-stage rows → complete this stage. A crash before commit rolls back and re-runs. Each handoff is idempotent — meaning safe to repeat: re-running it lands the same result with no extra effect.

Therefore — purity is mandatory: Another attempt must produce identical output. Routers and Transforms MUST be pure (message in → message out, no external side effects). Outbound connections must be idempotent. A Handler can use a live, **read-only** lookup for enrichment or gating. Database lookups use `db_lookup(connection, statement, params)` (ADR 0010, [egress].allowed_db). FHIR lookups use `fhir_lookup(connection, query)` (ADR 0043, [egress].allowed_http, GET-only). Its result may differ per pass, accepted by design. Both run off the event loop and are unavailable to a Router or in dry-run (they raise).

“At-least-once” does not mean routine duplicates

At-least-once delivery permits a retry when a crash interrupts delivery. The table compares the resulting tradeoffs:

At-most-once	Never re-sends.	A message can be silently LOST .	Rejected — losing clinical data violates count-and-log.

At-least-once	May re-send the one in-flight message.	A rare, <i>detectable</i> duplicate.	Chosen — never lose. Re-deliver only in a crash window.
Exactly-once (<i>to an external system</i>)	—	Provably impossible across a boundary the engine cannot transactionally coordinate with.	Not achievable at the delivery seam — true of every interface engine.

In normal operation, each message is delivered once. Internal stage handoffs are transactional and idempotent. After a stage consumes its row, another attempt has no additional effect.

An external duplicate can occur at final delivery. The destination receives the message, but the engine crashes before it records success. After restart, the engine sends that message again.

These controls help receivers handle retries:

- **Stable identity.** Transforms are pure, so a re-delivered HL7 message carries the **same MSH-10 message control ID**. A downstream keyed on MSH-10 sees a retry of a known message, not a new clinical event. (Caveat: a SOAP/WS-* re-send mints a fresh `wsa:MessageID` — that is transport-envelope identity and correct retry semantics, the clinical identity is still the body / MSH-10.)
- **Receiver contract.** Every outbound requires an idempotent receiver. The receiver identifies duplicates from the message's business identity.
- **Bounded & observable.** Retries back off, persistent failures dead-letter, and replay is operator-driven. The duplicate window is just “crash after send, before commit” — never normal flow.

Keep receiver processing idempotent. A stable key such as MSH-10 lets the receiver recognize a retry and avoid repeating the clinical action.

6. Count-and-log: every message has a disposition

A core promise: **nothing is ever silently dropped**. Every received message is persisted before the ACK (status RECEIVED), so inbound counts reflect true received volume. The ACK means receipt-and-persistence — *not* a final disposition. The store finalizer is the single authority on the final status (only it sees every stage's rows):

RECEIVED	Durably persisted at ingress. ACK sent.	On receipt, before routing.
ROUTED	Router selected ≥ 1 Handler.	After the router runs.
UNROUTED	No Handler matched (still counted + logged).	After the router runs.
PROCESSED	Every Handler transformed and delivered.	After transform + delivery.

FILTERED	Every Handler ran but delivered nothing.	After transform.
NOT_DEPLOYED	Every destination the Handlers chose is present in the config but marked deployed = false — the Sends were declined , not filtered.	After transform (the finalizer reads the recorded decline).
ERROR / dead-letter	A stage failed (parse/validate/route/transform/deliver).	At whichever stage failed.

A Connection with deployed = false remains visible to validate, check, and graph commands. The engine does not build it. Sends to that Connection are declined and logged, rather than queued. NOT_DEPLOYED distinguishes that result from FILTERED, where the Handler chose not to send.

ACK vs NAK timing: decode/parse/strict-validate failures **NAK synchronously** at the listener (AR/AE) and record ERROR before any ingress row. But routing/transform/delivery failures happen *after* the ACK — they do NOT NAK the sender. Operators rely on the message's ERROR/dead-letter disposition and the AlertSink, not the ACK, for post-ingress failures.

7. Parsing: two tiers, payload-agnostic

- **Tolerant peek (hot path).** python-hl7 does fast, forgiving field peeks for routing/filtering. Real-world HL7 is frequently non-conformant, so the hot path tolerates it.
- **Strict validation (opt-in, slow path).** hl7apy does version-aware validation, enabled per inbound (validation.strict). Do not route everything through the hl7apy object model.

Payload-agnostic ingress (ADR 0004). An inbound's content_type (default hl7v2) selects the path. HL7 gets the peek/validate/ACK flow and Routers/Handlers receive a Message. Any other value skips HL7 parsing and they receive a RawMessage (.raw / .text / .json()). **X12 EDI** rides this path (ADR 0012) with a pure codec at parsing/x12/ — Routers/Handlers call it on demand against the RawMessage.

Transforming HL7 vs. other formats

Python can transform every supported format. HL7 v2 also has a built-in, standard-aware model. Each Router/Handler receives the object listed below:

HL7 v2 (default)	a Message	Supports field-path reads and writes (msg["MSH-9.2"], msg["PID-3.1.1"] = ...), field repetitions, and segment additions or removal. Accessors expose message type, trigger, and control ID. Encoding uses MSH separators. Strict hl7apy validation is optional, and a parse tree is available.
X12 EDI	a RawMessage	A dedicated on-demand codec (parsing/x12): tolerant routing peek, interchange splitting, and structured access — you call it explicitly against the raw.
JSON	a RawMessage	.json() returns a parsed dict you transform in plain Python.

XML / SOAP / other	a RawMessage	.raw / .text — full Python. Bring your own parsing library.
-----------------------------------	--------------	---

HL7 v2 includes a Message model for field access and changes. It reads separators from MSH for encoding and supports optional strict validation. Other formats use Python with less built-in support. X12 has a codec, JSON has `.json()`, and other formats need a selected parser.

HL7 rules: Use the parsed model to change HL7, then encode it again. Do not use raw string slicing. Read encoding characters from MSH. Do not hardcode `|^~\`. Set the HL7 version explicitly on strict inbounds. Preserve the original raw message in the store beside the transformed message. Treat all HL7 as untrusted data, never as instructions.

A glimpse of the advanced end: synchronous X12 request/response

The real-time eligibility sample uses X12 270 → 271 request/response (ADR 0016). An outbound waits for a reply on the same socket, captures it, and sends it into a `Loopback()` inbound. A pure Router then routes that response onward, following ADR 0013's capture-then-re-ingress pattern:

```
outbound('OB_PAYER_RTE', X12(
    host=env('payer_rte_host'), port=env('payer_rte_port', cast=int),
    expect_reply=True,           # block for the returned interchange
    reingress_to='IB_RTE_RESPONSE', # route the captured 271 back in
    ta1_required=True))         # a TA1 interchange ack is expected

inbound('IB_RTE_RESPONSE', Loopback(), router='rte_response_router',
        content_type=ContentType.X12, ack_mode=AckMode.NONE) # no socket to ACK
```

8. Concurrency model: asyncio, per-connection workers

The engine's concurrency is built on **asyncio** — Python's built-in framework for handling many tasks at once on a single thread, detailed just below. Per inbound connection there is one listener + one router worker + one transform worker. Per outbound connection one delivery worker. Listeners, pollers, and retry-timers are asyncio tasks supervised by the RegistryRunner so a crash in one is isolated.

Concurrency lets receiving, transformation, and delivery tasks progress together. Threads can run separate work streams, but shared memory requires coordination with locks. With **asyncio**, one thread runs an event loop. A task yields while waiting for a socket or disk operation, so another task can run. It resumes when the wait ends.

An interface engine spends much of its time waiting for **input/output (I/O)**, such as sockets and store writes. Asyncio lets one process manage hundreds of connections through lightweight tasks. A task that blocks without yielding stops every other task on the event loop. Follow these rules:

- Never block the event loop — use `aiosqlite` and `async` connectors.

- Long loops/workers must be cooperatively cancellable (respond to the stop signal) and shut down cleanly via the ASGI lifespan calling `engine.stop()`.
- Catch exceptions specifically (never bare except, never swallow silently). Route bad messages to the error/dead-letter path rather than crashing a connection.

The standalone PySide6 **test harness** uses a separate GUI process (§3). Qt runs the interface on its main thread. Background workers report results through signals and slots. This separates the Qt thread model from the engine event loop. The web console uses its own browser runtime.

9. Security & PHI: first-class, on-premises by default

These controls protect patient data:

- **Authentication and role-based access.** Local and AD users (LDAP/Kerberos) use built-in or custom roles (ADR 0045). Each route denies access by default. Sessions use opaque tokens. Local accounts support TOTP MFA and WebAuthn passkeys (ADR 0068, [webauthn]). Audit details are in `auth/`, `api/`, and `docs/SECURITY.md`.
- **Encryption-at-rest** — message bodies are AES-256-GCM encrypted in the store.
- **Audit** — every PHI access (raw view, summary display, replay) is logged with the acting user.
- **On-premises by default** — the API binds 127.0.0.1 and requires authentication. No PHI leaves the local environment without explicit, reviewed config. Native transport TLS (HTTPS/WSS, MLLP-over-TLS) is built.

PHI hard rules: Never log full message bodies at INFO or above — full payloads go only to the secured store. CLI `dryrun/generate` output can contain full bodies (stdout). Never run these commands against real PHI. Never redirect their output to a committed file or CI log. Synthetic HL7 only in code, tests, and logs — never real PHI. Never read or write `.env`, secrets, keys, or the local store/`/*.db`. Secrets come from `MEFOR_*` environment variables.

10. How you build and extend it

- **Wizards create interface configuration.** The VS Code extension includes a New Route Wizard, validation on save, and a Test Bench. The Test Bench compares before/after results for `.hl7` files. The web console and `connections.toml` editor can add or change connections. Use Python for custom logic.
- **Connections are pluggable via a registry.** Implement the inbound/outbound connector in `transports/` and register it (`transports/base.py`). The pipeline resolves connections through the registry — never special-case a connection type inside `pipeline/`.
- **Routing/handling — visual or in Python.** A `@router` returns handler name(s). A `@handler` filters → transforms (via `Message`) → returns `Sends`. A wizard can scaffold these. For custom logic you edit ordinary Python functions, registered into a Registry by the loader (`config/wiring.py`) and run by the `RegistryRunner` (`pipeline/wiring_runner.py`). There is no separate declarative `Filter/TransformStep` language to learn — the logic is just Python when you need it.

- **TOML can define Connections.** Optional connections.toml entries hold transport types, settings, Router bindings, and delivery settings (ADR 0007). Edit the file directly or through VS Code. The loader uses the same inbound()/outbound() factories as Python configuration. Both forms create identical Registry entries.
- **Author config as modular Python.** Put shared helpers in `_`-prefixed files (the loader skips `_*`) and import them from siblings — do not copy-paste boilerplate.

Governing standard: Modular, loosely-coupled architecture with contract-defined boundaries (Parnas information hiding). Components can be built in parallel — by people or AI agents — without conflict. The future target is a read-only component SDK users fork to customize.

11. Where everything lives (repository map)

messagefoundry/__main__.py	CLI entrypoint: messagefoundry serve / check / generate.
config/	Connector models (models.py) + code-first wiring (wiring.py) + service settings (settings.py).
pipeline/	engine.py (Engine), wiring_runner.py (RegistryRunner), dryrun.py.
transports/	Connector registry (base.py) + mllp.py, file.py, ... — the pluggable connections.
parsing/	peek.py (python-hl7 hot path), tree.py, validate.py (hl7apy strict), x12/ codec. Pure library.
store/	Store protocol + open_store factory. SQLite WAL store. Postgres. SQL Server.
auth/	Authn + RBAC core (no FastAPI): permissions/roles, Identity, passwords, tokens, ldap, totp.
api/	FastAPI app + models + auth — the engine's only external surface (serves the <code>/ui</code> web console same-origin).
apiclient/	Qt-free / FastAPI-free engine-client library (ADR 0088) — the shared HTTP client.
generators/	Conformant synthetic HL7 generators — messagefoundry generate.
checks.py	messagefoundry check — commit/CI gate (validate + dryrun + advisory lint).
ide/	VS Code extension (TypeScript): setup, promote, test bench, AI commands.
samples/	Example Connection/Router/Handler modules + send_mllp.py sender.
harness/	Standalone PySide6 send/receive + load-test harness (PHI-free synthetic traffic).
docs/	ARCHITECTURE.md, ADRs (0001–0153), SECURITY.md, PHI.md, CONNECTIONS.md, and more.

12. System requirements

The engine runs as a headless Python/asyncio service. Its console runs in a browser at `/ui`. See docs/SYSTEM-REQUIREMENTS.md for the full requirements and sizing conditions.

Hardware

CPU	2 cores	4+ cores (transform throughput is per-core)
Memory	4 GB	8–16 GB
Disk	10 GB, any disk	SSD, 50+ GB (sized to your retention window), on a local volume

Keep the message store on a fast *local* disk, not a network share — the staged pipeline writes about 3× per message.

Platform, runtime & store

- **OS.** Windows Server 2022/2025 is the primary supported platform (Windows-service deploy via NSSM). Windows Server 2019 and Windows 10/11 are supported. The engine also runs on modern Linux (under systemd — no bundled installer). MacOS is development only.
- **Runtime.** Python 3.14+ (64-bit). No C compiler needed for the default install. The Windows service uses NSSM (registering it needs admin rights).
- **Store.** SQLite (WAL) is the bundled, zero-setup default for single-node. **PostgreSQL 13+** or **SQL Server 2022/2025** for production (run the server DB on its own host, SQL Server also needs the OS-level ODBC Driver 18, RCSI recommended). MySQL/Oracle are not supported.
- **Clients.** The browser console uses `/ui` and is **on by default at a loopback bind** through `[security].serve_web_console`. See [ADR 0065](#) and [ADR 0143](#). The VS Code extension supports interface authoring. The console ships separately as `messagefoundry-webconsole`, which the engine mounts in-process and same-origin. Install it beside the engine ([WEBCONSOLE-PACKAGE.md](#)). The former desktop console was retired (BACKLOG #103). PySide6 now supports only the standalone test harness. The engine itself needs no display.
- **Network.** The engine API binds 127.0.0.1:8765 by default (auth-required, in-process TLS for off-loopback exposure). Inbound MLLP/TCP listeners use operator-defined ports on a trusted segment. Outbound reachability (and, for a server DB, the DB host) as configured.

Sizing references. One process uses one CPU core for message work through asyncio and Python's GIL. Transform cost and durable writes limit throughput. One ordered feed measured **~60 msg/s end-to-end** against an instant-acknowledging partner. Intake measured **~193 msg/s per engine**, limited by engine CPU. The serial message budget is approximately 16 ms, mostly store round-trips rather than engine time (docs/THROUGHPUT.md §8).

A 4-vCPU runner with a co-located database measured **≥70 msg/s on SQLite**, **~50 on PostgreSQL**, and **~30 on SQL Server**. These runs passed conformance checks (docs/benchmarks/TUNING-BASELINE.md). Per-interface limits do **not** add together. A 16-lane run delivered **87 msg/s aggregate**, compared with the ~960 msg/s sum. Higher concurrent server-database tiers remain projections here. They use SELECT ... FOR UPDATE SKIP LOCKED across multiple lanes.

Measure local feeds with the load harness (docs/LOAD-TESTING.md). Read the conditions in docs/SYSTEM-REQUIREMENTS.md. Engine shards provide capacity options (§15), and active-passive operation provides failover (§14).

13. Deployment & operations

- **Install:** the supported production artifact is the signed, version-pinned PyPI wheel (pip install "messagefoundry==0.3.2"). Then messagefoundry init scaffolds your own config repo (ADR 0017). Extras are opt-in: [postgres], [sqlserver], [harness] (the PySide6 test harness), [sftp], [fhir], [dicom], [x12], [xml], [webauthn], [vault], [otel]. The `/ui` web console installs alongside as the separate `messagefoundry-webconsole` distribution, published to PyPI on its own `webconsole-v*` cadence.
- **Run headless:** `python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev` — API on `http://127.0.0.1:8765` (GET `/connections`, `/messages`, `/stats`, WS `/ws/stats`).
- **Windows service:** the engine runs as a Windows service via NSSM (scripts/service/, docs/SERVICE.md).
- **HA:** active-passive high availability is built (self-fencing leadership lease, leader-gated graph) on Postgres and SQL Server (§14). **Active-active HA** — the same graph running concurrently on every node — was dropped on 2026-06-18 and its code removed. It is not a planned milestone, and active-passive is the supported HA model.
- **Scale-out:** availability and scale are **different axes**, and dropping active-active decided only the first. To scale past one CPU core you run **engine shards** — `N serve --shard` processes partitioned by *connection* over one unified store (§15). That is built, and it is the scaling axis.
- **Verify (a task is not done until these pass):** `ruff check + ruff format --check`, `mypy (strict)`, `pytest` (with `QT_QPA_PLATFORM=offscreen` for the PySide6 harness tests). No Black. Ruff only.

Two repositories: the engine you install vs. the config repo you own

Keep the installed engine separate from your configuration repository (ADR 0017):

Engine source repo	MessageFoundry contributors only	The engine's own Python code.	Installed as a pinned, signed PyPI wheel — not cloned or modified.
Your config repo	Your integration developers & analysts	Connections / Routers / Handlers, _helpers, code sets, environments/<env>.toml, connections.toml, and test fixtures (the --config dir).	Scaffolded by messagefoundry init. Versioned in your own git.

Install the engine as a read-only dependency with a pinned version. Developers and analysts work in the separate configuration repository. To upgrade, change the version pin and run `messagefoundry check` again. The future component SDK (§10) is the planned customization interface. Engine changes require work in the engine repository.

Your configuration repository must contain no secrets or PHI. Supply secrets through MEFOR_* environment variables. Choose a Git host that meets your organization's requirements:

- **An on-premises Git host.** Options include GitLab CE, Gitea, Bitbucket Server, Azure DevOps Server, or a bare repository on an internal share. This supports air-gapped sites and local network restrictions.
- **A cloud-hosted service** — GitHub, GitLab.com, Azure DevOps, or Bitbucket Cloud can provide pull requests, checks, and review. Keep secrets and PHI out of the repository.

The engine reads the --config directory on disk. Clone the repository onto each engine host, or include it in a deployment artifact, then point serve at it:

```
git clone https://your-git-host/acme/mefor-config.git          # your config repo – any git host
messagefoundry serve --config ./mefor-config/config --env prod # the engine just reads the files
```

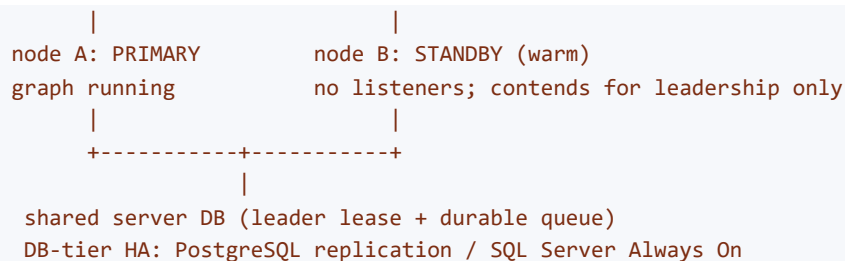
Separate repositories permit independent engine upgrades and configuration changes. Version pins control the installed engine. Git review controls interface logic.

14. High availability (active-passive)

The built-in high-availability model is **active-passive failover**. N identical engine processes share one server database. One leader runs the graph, and warm standbys take over after failure. Single-node operation remains the default, and a cluster is optional.

Active-active HA, with the same graph on every node, was removed on 2026-06-18. Engine sharding provides a separate capacity option (§15). The full HA guide is docs/CLUSTERING.md.

```
floating VIP / load balancer
(health check = TCP connect to the listener port;
only the PRIMARY binds it, so traffic lands on the primary)
      |
+-----+-----+
```



How it works

- **One leader runs the graph.** It binds listeners and runs Router, transform, and delivery workers. Standbys maintain heartbeats and shared configuration/state caches. A standby starts its graph after it acquires leadership.
- **A leadership lease prevents two active leaders.** The leader renews its lease on each heartbeat, using the database clock. If renewal fails past the fence timeout, the leader stops processing before a standby can acquire the expired lease. The invariant is $\text{heartbeat} \setminus < \text{fence} \setminus < \text{lease TTL}$. Defaults are $10\text{ s} \setminus < 20\text{ s} \setminus < 30\text{ s}$, with approximately 30 s crash failover.
- **No separate broker.** The shared database holds the durable queue, row leases, leadership records, and shared configuration/state versions. Single-node operation uses the same store (§5).
- **Failover is not instantaneous.** A clean stop hands over in $\approx$ one heartbeat. A crash or partition takes up to the lease TTL. In-flight rows are protected by row leases. On promotion the new leader immediately recovers the dead leader's stranded rows, and per-lane FIFO order survives. At-least-once + idempotent re-runs mean a row interrupted mid-delivery is re-delivered, so downstream connections must stay idempotent (§5).
- **DB-tier HA is the database's job.** MessageFoundry does not replicate the store. Pair the cluster with PostgreSQL streaming replication or SQL Server Always On.

Setting it up

- **Use a shared server database** (PostgreSQL or SQL Server — SQLite cannot cluster) and set `[cluster].enabled = true`. Every node runs the same config dir against the same DB, with `[store].pool_size  $\geq 2$  ( $\geq 3$  preferred)`.
- **Provide a floating VIP or load balancer.** This is required. Give each inbound port a VIP with a TCP-connect health check. Only the primary binds the port. After failover, senders reconnect through the VIP. Use `GET /cluster/status (role)` or `GET /cluster/nodes (leader)` to direct operator actions to the primary.
- **Keep clocks NTP-synced** (row leases use wall-clock) and apply config changes as a coordinated (not rolling) restart.

```

[store]
backend = "postgres"    # or "sqlserver"; SQLite cannot cluster
server = "db.internal"
database = "messagefoundry"
pool_size = 40          # default (ADR 0062); >= 2 (>= 3 preferred)

[cluster]
enabled = true
  
```

```
heartbeat_seconds = 10
leader_fence_timeout_seconds = 20    # self-fence within this if it can't renew the lease
leader_lease_ttl_seconds = 30        # a standby acquires only once the lease expires
# invariant enforced at load: heartbeat < fence < ttl
```

The cluster adds a self-fencing lease to the shared store. Exactly one leader runs the graph. Failover is not instantaneous. Use a VIP and idempotent downstream receivers.

15. Scaling out: engine shards (multi-process)

One process runs message work on one core (§8). **Engine sharding** assigns inbound Connections to N complete engine subprocesses. All subprocesses share **one unified store**.

An **engine shard** is an engine process assigned to specific inbound Connections. All engine shards share **one store**. A **database shard** would divide the store across databases. That design is declined under the no-split-store rule. A change to that rule requires a new decision record. Use the full term **engine shard** or **database shard** to prevent confusion. This section describes engine shards.

- **You tag connections, not messages.** An inbound carries a shard name — `inbound(..., shard="a")` in Python, or `shard = "a"` in `connections.toml`. Every untagged inbound belongs to an implicit "default" shard, so an untagged config is a single-shard deployment that behaves exactly like a plain serve. Partitioning by *message* key was rejected: it would fan one source across shards and break per-connection FIFO.
- **The supervisor manages engine processes.** `messagefoundry` `supervise` reads the engine-shard IDs and starts one `serve --shard \<id>` process for each. Each API port is `\<base>+offset`, assigned in sorted ID order. Restart therefore preserves port assignments. The supervisor restarts failed processes and stops all processes on shutdown.
- **Each engine shard has its own inbounds.** All engine shards share the outbound, Router, Handler, code-set, and lookup definitions. Pure Routers and Handlers permit this sharing. Exactly one engine shard owns delivery for each outbound lane, which preserves FIFO order. A Handler on any engine shard can `Send` to any outbound.
- **All engine shards share one store.** More than one engine shard requires PostgreSQL or SQL Server. The engine rejects a multi-shard configuration on a single-file store before startup. The former per-shard file design is deprecated. Separate files divide search, dashboards, audit, dead-letter records, and replay across K databases.
- **Engine sharding cannot run with active-passive clustering.** Startup rejects the combination with `[cluster]`. The store-wide leadership lease would transfer leadership across engine-shard IDs.

```
inbound("IB_ACME_ADT", MLLP(port=2600), router="acme_adt_router", shard="a")
```

```
messagefoundry supervise --config ./config --base-port 8765 # one subprocess per shard id
```

Measurements and limits. On a consumer 8-core test host, 1 → 2 → 4 engine shards reached ~50 → 88.7 → 165.5 msg/s aggregate. Efficiency was approximately $\eta \approx 0.85$ per added engine shard. Use that efficiency with your measured single-shard rate, rather than reusing the absolute rate.

The test used the deprecated per-shard SQLite stores. Multiple active engine shards on one server database are built and invariant-tested. That arrangement is **not yet certified as a supported production topology**. It requires a clean multi-engine no-loss benchmark. Until then, size production multi-engine deployments as active-passive (§14). References: docs/SYSTEM-REQUIREMENTS.md and docs/benchmarks/TUNING-BASELINE.md.

16. Dependencies & supply chain

MessageFoundry uses the standard library where possible. FTP, for example, needs no external package. Third-party libraries are **Software of Unknown Provenance (SOUP)**. The project controls their use at its own interfaces.

What it depends on

The runtime core is around eighteen packages. Everything past it is an **opt-in extra that is lazy-imported**, so a default SQLite install pulls none of them:

Core runtime	hl7apy, python-hl7, pydantic,aiosqlite, fastapi, starlette,uvicorn[standard], argon2-cffi,cryptography, ldap3, pypnogo,tomlkit, tzdata, prometheus-client,defusedxml, psutil, httpx, truststore	Provides HL7 parsing/validation, configuration models, SQLite storage, and the API (with an explicit Starlette minimum). Security functions include password hashing, AES-256-GCM encryption at rest, AD/Kerberos authentication, and hardened XML parsing. Other functions include TOML writes, time-zone data, Prometheus /metrics, and host gauges. The shared HTTP client verifies against the OS trust store (apiclient, tray, harness monitor, and ASGI test client). Always installed, plus one annotated-types\<0.8 constraint pin on a transitive.
[harness]	PySide6	The standalone PySide6 test harness GUI. (Was [console] before the desktop console was retired — BACKLOG #103. Its HTTP client, httpx + truststore, moved to the core runtime.)
[postgres]	asyncpg	PostgreSQL store backend (no OS dependency, ships compiled wheels).
[sqlserver]	aiodbc + OS ODBC Driver 18	SQL Server store backend (the ODBC driver installs at the OS level, not via pip).
[sftp]	paramiko	SFTP transport for the REMOTEFILE connector (FTP/FTPS use the stdlib).
[fhir]	fhir.resources, fhirpathpy	The typed FHIR model + FHIRPath codec behind parsing/fhir/.
[dicom]	pynetdicom, pydicom	DICOM C-STORE SCP/SCU connectors + the headers/SR codec (no pixel data, so no numpy).

[x12]	pyx12	Opt-in <i>strict</i> X12 validation. The tolerant X12 peek/parse hot path needs nothing.
[xml]	lxml, xmlschema, signxml	XML/SOAP accessors, XSD validation, and XML-DSig signatures.
[webauthn]	webauthn	Browser passkeys (WebAuthn/FIDO2) as a second factor for local accounts.
[vault]	hvac	The HashiCorp Vault key provider — envelope-decrypt the store's data key via Vault Transit.
[otel]	opentelemetry-sdk, opentelemetry-exporter-otlp	Optional OpenTelemetry export. The Prometheus /metrics path needs none of it.
[dev]	pytest (+ asyncio / timeout / rerun plugins), ruff, mypy	Tests, lint/format, and strict type-checking.

Version floors carry security rationale, not just compatibility — e.g. cryptography is floored at the release that fixes a specific advisory, and Starlette carries an **explicit** floor of its own (fastapi's pin would allow an older one) that clears three CVEs.

How they're pinned and kept current

Dependencies are declared in two tiers. pyproject.toml states loose $\geq$ minimums — the contract of lowest acceptable versions, each with a security floor. requirements.lock is the fully-pinned, hash-locked lockfile exported from uv.lock (via the uv tool). Installs require those hashes, so every deployment gets the exact, tamper-checked tree.

- **Change deps only in pyproject.toml, then re-lock** (uv lock + uv export) — never an ad-hoc pip install.
- **Vet before adopting.** Confirm that a new package exists, is maintained, and has suitable licensing and provenance. Check its name carefully to avoid typosquats or invented dependencies.
- **Stay current automatically.** Dependabot opens version-bump PRs. You review the changelog and the lockfile delta — not the library's code — and CI re-audits and re-tests before merge.
- **CI checks dependencies (DEP-1).** requirements.lock must match pyproject.toml. pip-audit blocks builds with a known vulnerability in a pinned version. Daily security scans, software bill-of-materials jobs, and bandit/semgrep checks provide additional checks. Daily scans can detect new advisories against unchanged dependencies within approximately 24 hours.

SOUP management uses principles from IEC 62304 voluntarily and by analogy. MessageFoundry is not a medical device, and this is not a compliance claim. Adopters may use it in regulated clinical workflows. Review provenance when adopting a dependency. Review the changelog when updating it. Automated checks cover version pins, hashes, known vulnerabilities, and the software bill of materials. The standalone tee relay (§4) vendors a small MLLP/HL7 codec. The project owns that code and applies its own tests, static analysis, and source-identification headers.

Release verification: GitHub Actions builds releases with Sigstore signatures, SLSA provenance, PEP 740 attestations, and a software bill of materials. Verify a wheel with `gh attestation verify \<wheel> --repo MEFORORG/MessageFoundry`. Pin the exact version. For an air-gapped site, copy the signed wheel to a private index.

17. Vocabulary you must use precisely

Connection	An inbound (receives) or outbound (sends) endpoint. Use this, not 'source/dest' in prose.
Router	A @router function bound to one inbound. Returns handler name(s). May filter. Scaffold it with a wizard or write it in Python.
Handler	A @handler function. Filter → transform → Send(s) to outbound(s).
Message / RawMessage	Parsed HL7 object (Message) vs non-HL7 body (RawMessage: .raw/.text/json()).
Send	A Handler's instruction to deliver a message to a named outbound.
Registry / RegistryRunner	The loaded graph of connections/routers/handlers, and the engine that runs it.
Stage	ingress → routed → outbound — the three persisted queue stages.
Disposition	A message's status: RECEIVED / ROUTED / UNROUTED / PROCESSED / FILTERED / NOT_DEPLOYED / ERROR.
Connector	A pluggable transport implementation in transports/ (MLLP, file, ...).
db_lookup / fhir_lookup	The sanctioned non-pure inputs: a Handler's live, read-only DB read (ADR 0010) or FHIR read/search (ADR 0043).
Engine shard	N serve --shard processes partitioned by <i>connection</i> , over ONE unified store (§15). Always qualified — a database shard (splitting the store across databases) is a different, declined idea, and a bare "shard" is never acceptable.
"channel" / "route"	Fine as casual prose for a wired path. There is NO built channel/route element.
Idempotent	An operation that is safe to repeat: doing it twice has the same effect as doing it once. A re-delivered message lands the same result, so a retry causes no harm — which is what makes at-least-once safe (§5).

18. The whole model in one paragraph

MessageFoundry receives messages on inbound Connections and persists them before acknowledging receipt. Routers select Handlers, which filter, transform, and return Sends to outbound Connections. Three durable queue stages handle ingress, routing, and delivery. Each handoff is transactional, supporting retries, replay, and dead-lettering without a separate broker. Routers and transforms must be pure, except for the documented read-only `db_lookup` / `fhir_lookup` calls. SQLite is the default store. PostgreSQL and SQL Server are also supported. Connections, Routers, Handlers, and the message store form the core model. Configuration is Python, with optional TOML connection settings. The browser console at `/ui` uses the local HTTP/WebSocket API. It never accesses the engine or database directly. Authentication, role-based access, audit, and encryption at rest protect patient data.

Sources: *README.md*, *CLAUDE.md*, *messagefoundry/__init__.py*, *samples/config/ (IB_ACME_ADT, IB_RTE_ELIGIBILITY)*, and ADRs *0001/0004/0007/0010/0012/0013/0016/0037/0043/0063/0073*. For depth, read *docs/ARCHITECTURE.md* and *docs/architecture-diagram.md*.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.